

FENIXCMS_7

FASE 7 - ELIMINACION DE LA SEGUNDA REALIDAD, POSTGRESQL-ONLY Y CONTEXTO TENANT CONFIABLE

Documento tecnico de implementacion para AI Studio.

Repositorio: `fenixcmssl-spec/moreno`

Rama: `main`

Ultimo commit auditado: `843ac4dd7cf19831363859ae45351b052ac9dc71`

Prioridad: CRITICA - bloqueador de produccion por integridad de datos y aislamiento.

0. HALLAZGO MAXIMO

La auditoria actual demuestra que el backend PostgreSQL no es todavia la unica realidad del sistema.

En `lib/storeContext.tsx` siguen coexistiendo:

- Firebase/Firestore.
- `INITIAL\_\*` como estado inicial del negocio.
- `localStorage` para estado de autentificacion de UI.
- mutaciones de React state que no llegan a PostgreSQL.
- fallbacks locales despues de errores de API.
- `resetToDemoData()` dentro del runtime.

En `app/api/storefront/route.ts` y `app/api/storefront/resolve/route.ts` el cliente todavia puede enviar `host`, `slug`, `store` o `tenant` por query. En produccion eso no debe elegir otro tenant.

Por tanto la prioridad maxima es convertir FenixCMS en:

`PostgreSQL + APIs server-side + sesion real + PayPal real + hostname confiable = unica fuente de verdad.`

1. OBJETIVOS DE FENIXCMS_7

1. Eliminar Firebase/Firestore del runtime de produccion.
2. Convertir StoreContext en una capa de UI/cache y nunca en una base de datos.
3. Eliminar `INITIAL\_\*` del camino runtime de produccion.
4. Hacer que todas las mutaciones de negocio pasen por APIs autorizadas.
5. No confirmar en la UI una mutacion hasta que PostgreSQL confirme.
6. Eliminar fallback local despues de errores de API.
7. Quitar autoridad a `localStorage` sobre identidad, role y tenant.
8. Bloquear tenant spoofing mediante query parameters en storefront.
9. Mantener el aislamiento multi-tenant desde PostgreSQL.
10. Añadir tests y un production gate que detecten regresiones.

2. EVIDENCIA ACTUAL AUDITADA

2.1 Firebase sigue en runtime

`lib/firebase.ts` inicializa Firestore y `lib/storeContext.tsx` importa `db`, `doc` y `getDoc`.

El StoreContext intenta leer `tenants/tenant\_demo` mediante Firestore.

2.2 StoreContext arranca con datos demo

Actualmente inicia estado con:

- `INITIAL\_APPLICATIONS`
- `INITIAL\_PLANS`
- `INITIAL\_LICENSES`
- `INITIAL\_TENANT`
- `INITIAL\_PRODUCTS`
- `INITIAL\_ORDERS`
- `INITIAL\_BLOG\_POSTS`
- `INITIAL\_CLASSIFIED\_ADS`
- `INITIAL\_MEDIA\_ITEMS`
- `INITIAL\_PLUGINS`

```
- `INITIAL_THEMES`  
- `INITIAL_MARKETPLACE_ITEMS`
```

2.3 Hay CRUD solo en memoria

Se han encontrado mutaciones como `createPlan`, `updatePlan`, `deletePlan`, `createLicense`, `updateTenant`, `addProduct`, `updateProduct`, `deleteProduct`, `addBlogPost`, `updateBlogPost`, `deleteBlogPost`, `createOrder`, `togglePlugin`, `installNewPlugin`, `installNewTheme` y otras que modifican principalmente React state.

Una mutacion comercial no puede considerarse persistida si PostgreSQL no la confirma.

2.4 Hay fallback despues de errores

Hay funciones que intentan API y, si falla, crean datos locales o continúan con state actualizado.

Patron prohibido en produccion:

```
~~~text  
API falla -> catch -> crear entidad local -> success  
~~~
```

Patron correcto:

```
~~~text  
API falla -> error -> conservar estado anterior -> mostrar unavailable/error  
~~~
```

2.5 Auth local no es autoridad

`fenix\_backend\_auth` puede existir como cache visual, pero no puede autorizar nada.

2.6 reset demo existe

`resetToDemoData()` restaura `INITIAL\_\*` dentro del runtime.

2.7 Storefront acepta query tenant context

`effectiveHost = queryHost || headerHost` y `effectiveSlug = querySlug || headerSlug`.

En produccion el query param no debe poder cambiar el tenant.

3. ARQUITECTURA FINAL

Runtime general

```
~~~text  
Browser  
-> Next.js API  
-> Service  
-> Prisma  
-> PostgreSQL  
~~~
```

Auth

```
~~~text  
Browser  
-> /api/auth/session  
-> httpOnly session cookie  
-> PostgreSQL  
~~~
```

Storefront

```
~~~text  
Request hostname  
-> middleware / trusted proxy  
-> trusted tenant context  
-> StorefrontService  
-> PostgreSQL Domain/Tenant  
~~~
```

Prohibido

~~~text

Browser -> Firestore

Browser -> INITIAL\_\*

Browser -> local business database

Query ?tenant=... -> select arbitrary tenant

~~~

4. FASE 7A - ELIMINAR FIREBASE DEL RUNTIME

Archivos:

- `lib/firebase.ts`
- `lib/storeContext.tsx`
- `firebase-applet-config.json`
- `package.json`

Acciones:

1. Eliminar imports de `firebase` y `firebase/firestore` del runtime.
2. Eliminar el efecto que consulta `tenants/tenant\_demo`.
3. Buscar `firebase`, `firebase/firestore`, `getFirestore`, `initializeApp`, `firebase-applet-config`.
4. Clasificar cada uso como test/dev/runtime.
5. Cuando no quede ningun uso legitimo, eliminar la dependencia `firebase` del package.json.
6. Regenerar lockfile con `npm ci`.

5. FASE 7B - STORESOURCE COMO UI/CACHE

`StoreContext` puede mantenerse temporalmente, pero solo como proyeccion de datos del servidor.

Regla:

`API -> PostgreSQL -> respuesta canonical -> React state`.

Nunca:

`React state -> success -> supuesto almacenamiento`.

6. FASE 7C - CAPA API CLIENT

Crear `lib/api/client.ts` o equivalente.

Responsabilidades:

- `credentials: include`.
- parse JSON seguro.
- comprobar `response.ok`.
- convertir errores HTTP a errores tipados.
- distinguir 401, 403, 409, 422, 500, 503.
- nunca devolver datos inventados.

Ejemplo conceptual:

~~~ts

```
export async function apiRequest<T>(url: string, init?: RequestInit): Promise<T> {
  const response = await fetch(url, { ...init, credentials: 'include' });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(data?.error || `HTTP ${response.status}`);
  return data as T;
}
```

~~~

Adaptar al codigo real del repositorio.

7. FASE 7D - APPLICATIONS

Conectar `createApplication`, `updateApplication`, `toggleApplicationStatus`, `deleteApplication` a las rutas server-side.

Regla de mutacion:

- request al backend;
- si 2xx: usar el objeto devuelto por backend;
- si error: no modificar state como success;
- mostrar error;
- mantener el objeto anterior.

8. FASE 7E - PLANS

Conectar:

- ``GET /api/admin/plans``.
- ``POST /api/admin/plans``.
- update/patch del plan.
- delete del plan.

No crear IDs reales con ``Date.now()`` en el cliente.

No utilizar ``setPlans(INITIAL_PLANS)`` como restauracion de negocio.

El precio, currency, status y entitlements deben ser server-authoritative.

9. FASE 7F - LICENSES Y TENANT

``createLicense``, ``updateLicense``, ``toggleLicenseStatus``, ``updateTenant`` y ``updateTenantBranding`` no deben crear estados permanentes solo en el navegador.

Una licencia real solo puede provenir de:

- provisioning SaaS validado;
- endpoint administrativo autorizado.

Un Tenant real solo puede ser creado por el backend.

10. FASE 7G - PRODUCTS

Conectar CRUD con ``/api/products`` y los servicios existentes.

No crear producto persistente solo con ``setProducts``.

No aceptar precio, stock o tenantId como autoridad del cliente.

Para importacion masiva, crear/conectar un endpoint transaccional. Si falta, la funcion debe permanecer deshabilitada en production.

11. FASE 7H - ORDERS

``createOrder()`` del StoreContext no debe generar una Order real en memoria.

Usar ``/api/orders`` y ``OrderService`` como autoridad.

La UI solo agrega el pedido a state despues de una respuesta server-side confirmada.

12. FASE 7I - BLOG Y CLASSIFIEDS

Blog y Classifieds deben aplicar:

``UI -> API -> PostgreSQL -> respuesta -> UI``.

Eliminar el patron catch->crear local.

Si falta un endpoint de blog para una mutacion, crear el endpoint antes de marcar la funcionalidad como persistente.

13. FASE 7J - MEDIA

``addMediaItem()`` no debe fabricar un ``MediaItem`` local.

Usar `/api/media`.

Solo despues de upload + DB correctos:

- actualizar React state.

Si storage falla o DB falla:

- no success;
- no URL falsa;
- no MediaAsset local.

14. FASE 7K - PLUGINS, THEMES Y MARKETPLACE

Las funciones:

- `togglePlugin`;
- `updatePluginConfig`;
- `installNewPlugin`;
- `installNewTheme`;
- marketplace create/update/delete;

deben persistir mediante backend.

Al recargar la pagina el estado debe volver desde PostgreSQL y coincidir con la UI anterior.

15. FASE 7L - INITIAL_* SOLO COMO FIXTURES

`lib/initialData.ts` debe considerarse fixture de test/dev, no fuente runtime.

El scan de production debe rechazar imports de `INITIAL\_\*` desde:

- StoreContext runtime;
- PaymentService;
- StorefrontService;
- SuperAdminService;
- AuthService.

Excepciones: tests y herramientas de development claramente aisladas.

16. FASE 7M - localStorage Y SESION

`fenix\_backend\_auth` no puede ser autoridad.

Preferencia: eliminarlo como fuente de UI y consultar `/api/auth/session` al iniciar.

Si se conserva como cache visual:

- nunca usarlo para role authorization;
- nunca usarlo para tenant selection;
- nunca usarlo para acceso a API;
- nunca usarlo como prueba de autenticacion.

Si `/api/auth/session` devuelve 401: usuario no autenticado.

Si devuelve 500/503: mostrar servicio no disponible.

17. FASE 7N - RESET DEMO

`resetToDemoData()` debe salir del runtime de produccion.

Preferencia:

- mover a devtools/test;
- o aislarlo en un modulo solo development.

Un usuario final no debe poder restaurar una base demo en la UI de produccion.

18. FASE 7O - STOREFRONT TENANT TRUST

Modificar:

- `app/api/storefront/route.ts`
- `app/api/storefront/resolve/route.ts`

En production ignorar para seleccion de tenant:

- `?host=`
- `?slug=`
- `?store=`
- `?tenant=`.

Usar solamente contexto de confianza generado por middleware/proxy:

- `x-resolved-hostname`;
- `x-tenant-type`;
- `x-tenant-slug`;
- `x-custom-domain`.

Los query params de simulacion pueden permanecer solamente en development y con `ALLOW\_DEV\_TENANT\_SIMULATION=true`.

19. FASE 7P - STOREFRONT SERVICE FAIL-CLOSED

Modificar `lib/services/storefront.service.ts`.

En production:

- no `fallbackSlug` controlado por cliente;
- no `INITIAL\_TENANTS`;
- no memory fallback;
- no first tenant;
- no demo tenant.

Resolucion:

`trusted hostname -> verified Domain/System Subdomain -> PostgreSQL Tenant`.

Si no existe tienda: 404.

Si DB falla: 500/503.

20. FASE 7Q - VERIFIED DOMAIN

Custom Domain solo puede resolver storefront si:

- existe en PostgreSQL;
- pertenece al Tenant correcto;
- esta verificado;
- esta activo;
- el Tenant esta activo.

Un dominio no verificado nunca debe resolver datos de una tienda.

21. FASE 7R - STOREFRONT CACHE

El cache de `StorefrontService` puede mantenerse como aceleracion.

Reglas:

- cache key canonica por hostname;
- payload con tenantId;
- aislamiento estricto;
- DB error no -> memory fallback;
- invalidacion despues de mutaciones relevantes.

Cache = aceleracion, no fuente de verdad.

22. FASE 7S - SAAS LANDING

Eliminar textos que digan que la tienda fue provisionada en Firestore.

Debe indicar la arquitectura real de FenixCMS sin prometer una tecnologia que ya no se usa.

Tambien revisar mensajes de UI que indiquen 'aprovisionando' antes de tener respuesta real del backend.

23. FASE 7T - ERROR STATE

La UI debe distinguir:

- loading;
- ready;
- empty;
- unavailable;
- forbidden;
- conflict.

`DB devuelve []` -> empty.

`DB timeout` -> unavailable.

Nunca convertir unavailable en empty/demo.

24. FASE 7U - TESTS DE PERSISTENCIA REAL

Crear `tests/fenixcms-fase7.test.ts`.

Test 1: crear producto por API -> reload -> sigue existiendo.

Test 2: limpiar localStorage -> sigue existiendo.

Test 3: reiniciar proceso -> sigue existiendo.

Test 4: borrar en PostgreSQL -> UI deja de mostrarlo tras nuevo fetch.

Test 5: update via UI -> DB refleja el cambio.

Test 6: API 500 -> state no cambia como success.

Test 7: API 503 -> estado unavailable, no demo.

25. FASE 7V - TEST FIREBASE CERO

Crear un scan que falle si runtime contiene:

- `firebase` imports;
- `firebase/firestore`;
- `getFirestore`;
- `initializeApp`;
- `firebase-applet-config.json`.

Solo permitir fixtures/documentacion de tests explicitamente excluidos.

26. FASE 7W - TEST INITIAL DATA CERO

Crear scan de production runtime que rechace:

- `INITIAL\_TENANT`;
- `INITIAL\_PRODUCTS`;
- `INITIAL\_ORDERS`;
- `INITIAL\_PLANS`;
- `INITIAL\_LICENSES`;
- `INITIAL\_APPLICATIONS`;
- `INITIAL\_PLUGINS`;
- `INITIAL\_THEMES`;
- `INITIAL\_MEDIA\_ITEMS`;
- `INITIAL\_MARKETPLACE\_ITEMS`.

27. FASE 7X - TEST TENANT SPOOFING

Desde hostname de Tenant A probar:

~~~text

GET /api/storefront/resolve?slug=tenantB

GET /api/storefront/resolve?tenant=tenantB

GET /api/storefront/resolve?store=tenantB  
GET /api/storefront/resolve?host=tenantB.fenixcms.es

~~~

Resultado obligatorio:

- se mantiene Tenant A si el hostname representa Tenant A;
- o se rechaza;
- nunca se devuelve Tenant B por query.

28. FASE 7Y - TEST ROLE SPOOFING

Modificar manualmente `localStorage`:

~~~json

```
{"role": "super_admin", "email": "attacker@example.com"}
```

~~~

Debe seguir sin conceder permisos administrativos.

Los endpoints determinan permisos usando la sesion real server-side.

29. FASE 7Z - TEST OFFLINE

Desconectar PostgreSQL/API.

Resultado:

- login -> error/unavailable;
- plans -> unavailable;
- products -> unavailable;
- admin -> unavailable;
- storefront -> 503/error;
- media -> error.

Nunca:

- demo users;
- demo tenant;
- demo payments;
- demo products;
- demo metrics.

30. FASE 7AA - ELIMINAR FIREBASE

Despues de que el scan Firebase pase:

1. eliminar `firebase` de package.json;
2. eliminar `firebase-applet-config.json` si ya no tiene ningun uso;
3. eliminar `lib/firebase.ts`;
4. `npm ci`;
5. `npx prisma generate`;
6. `npm run lint`;
7. `npm test`;
8. `npm run build`.

31. FASE 7AB - CI PRODUCTION GATE

Actualizar/crear `.github/workflows/production-gate.yml`.

Debe ejecutar:

~~~text

```
npm ci
npx prisma generate
npm run lint
npm test
npm run build
```

npm run audit:production

~~~

`audit:production` debe incluir scans Firebase, INITIAL_*, fake/mock URLs, hardcoded secrets y fallback local.

32. FASE 7AC - PACKAGE SCRIPTS

Añadir, cuando corresponda:

~~~json

"audit:production": "tsx scripts/audit-production-fallbacks.ts && tsx scripts/audit-production-runtime.ts"

~~~

Los scripts no deben imprimir secretos; solamente archivo y tipo de hallazgo.

33. FASE 7AD - DOCUMENTACION

Actualizar README/documentacion para decir:

- PostgreSQL = fuente de verdad;
- Firebase/Firestore = no utilizado en production;
- StoreContext = UI/cache only;
- auth = cookie/session + PostgreSQL;
- storefront tenant = trusted hostname/context;
- cache = aceleracion, no autoridad.

34. ARCHIVOS PRINCIPALES

Revisar como minimo:

~~~text

lib/storeContext.tsx  
lib/firebase.ts  
firebase-applet-config.json  
package.json  
app/api/storefront/route.ts  
app/api/storefront/resolve/route.ts  
lib/services/storefront.service.ts  
middleware.ts  
app/api/auth/session/route.ts  
app/api/products/route.ts  
app/api/orders/route.ts  
app/api/media/route.ts  
app/api/classifieds/route.ts  
app/api/admin/applications/route.ts  
app/api/admin/plans/route.ts  
lib/services/payment.service.ts  
lib/services/auth.service.ts

~~~

Crear o actualizar:

~~~text

lib/api/client.ts  
tests/fenixcms-fase7.test.ts  
scripts/audit-production-fallbacks.ts  
scripts/audit-production-runtime.ts  
.github/workflows/production-gate.yml

~~~

35. ORDEN DE IMPLEMENTACION

1. Auditoria + scans.
2. Eliminar Firestore del StoreContext.

3. Crear API client.
4. Migrar Applications/Plans.
5. Migrar License/Tenant/Branding.
6. Migrar Products/Orders.
7. Migrar Blog/Classifieds/Media.
8. Migrar Plugins/Themes/Marketplace.
9. Migrar auth UI a session server-side.
10. Sacar reset demo.
11. Cerrar tenant spoofing del storefront.
12. Eliminar Firebase dependency.
13. Tests.
14. Lint.
15. Build.
16. Re-auditoria.

36. PROMPT MAESTRO PARA AI STUDIO

~~~text

### INICIO FASE FENIXCMS\_7

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

VALIDACION:

ORDENADOR DEBIAN

NO DESPLEGAR:

No desplegar al VPS produccion hasta cerrar todos los criterios.

OBJETIVO MAXIMO:

Eliminar la segunda realidad de runtime. PostgreSQL debe ser la unica fuente de verdad. Firebase/Firestore no debe participar en production. StoreContext solo refleja respuestas del servidor. Todas las mutaciones de negocio deben pasar por API + PostgreSQL. El storefront publico debe usar hostname/contexto confiable y no query params para seleccionar tenant.

PASO 1 - AUDITORIA

Busca firebase, firebase/firestore, INITIAL\_, FALLBACK\_, localStorage, inMemory, tenant\_demo, Date.now/Math.random como ids de negocio, catch con setState, query tenant/store/slug/host.

Clasifica production/test/dev y no declares PASS antes de revisar.

PASO 2 - FIREBASE

Elimina imports runtime, sync Firestore, lib/firebase.ts y dependencia npm cuando no quede uso legitimo.

PASO 3 - STORECONTEXT

Transforma StoreContext en UI/cache. Ninguna entidad de negocio puede crearse como fallback local.

PASO 4 - API CLIENT

Crea lib/api/client.ts o equivalente. Toda mutacion usa API, respuesta server-side y luego state.

PASO 5 - MIGRACIONES UI

Conecta Applications, Plans, Licenses, Tenant, Products, Orders, Blog, Classifieds, Media, Plugins, Themes y Marketplace a sus APIs reales. Si falta endpoint, crealo o deshabilita la mutacion en production.

PASO 6 - AUTH

Elimina localStorage como autoridad. Usa /api/auth/session y cookie HttpOnly. API unavailable = estado unavailable, no login local.

PASO 7 - DEMO

Saca resetToDemoData del runtime production.

PASO 8 - STOREFRONT

En production ignora queryHost/querySlug/queryStore/queryTenant. Usa solo trusted hostname/context headers.

PASO 9 - STOREFRONT SERVICE

En production no uses fallbackSlug controlado por cliente, INITIAL\_TENANTS, memory fallback ni first tenant. Resolver trusted hostname -> verified domain/system subdomain -> PostgreSQL tenant.

PASO 10 - TESTS

Crea tests para persistencia real, DB failure, Firebase cero, INITIAL\_\* cero, localStorage role spoofing y tenant spoofing.

PASO 11 - CI

Ejecuta npm ci, npx prisma generate, npm run lint, npm test, npm run build y npm run audit:production.

PASO 12 - INFORME

Devuelve archivos creados, archivos modificados, dependencia Firebase eliminada, mutaciones migradas a backend, scans, tests, lint,

build y blockers restantes.

NO DECLARES PASS SI:

- Firebase sigue en runtime;
- INITIAL\_\* sigue siendo fuente de verdad;
- una mutacion continua siendo local;
- un error API produce success local;
- localStorage concede permisos;
- un query param puede cambiar tenant en production.

**FIN FASE FENIXCMS\_7**

~~~

37. CRITERIOS DE ACEPTACION

- [] Firebase/Firestore no existe en runtime production.
- [] `firebase` eliminado de package.json cuando no haya uso valido.
- [] StoreContext no crea entidades persistentes localmente.
- [] StoreContext no usa INITIAL_* como fuente de verdad production.
- [] Todas las mutaciones relevantes pasan por API server-side.
- [] Error API no genera success local.
- [] localStorage no concede permisos.
- [] `/api/auth/session` es la autoridad de sesion.
- [] resetToDemoData fuera de production.
- [] query params no pueden cambiar tenant en production.
- [] hostname/contexto confiable determina tenant.
- [] custom domains requieren verificacion.
- [] DB empty != DB unavailable.
- [] scan Firebase pasa.
- [] scan INITIAL_* pasa.
- [] tenant spoofing pasa.
- [] role spoofing pasa.
- [] offline behavior pasa.
- [] npm test pasa.
- [] npm run lint pasa.
- [] npm run build pasa.
- [] production-gate pasa.

38. DECISION DE PRODUCCION

FENIXCMS_7 no añade una funcion comercial nueva. Su objetivo es eliminar la contradiccion entre el estado del navegador y el estado real de PostgreSQL, y cerrar una via publica de seleccion arbitraria de tenant.

Antes de pasar al VPS debe demostrarse:

****lo que PostgreSQL conoce como verdadero es exactamente lo que el frontend y el storefront publican como verdadero.****

FIN DEL DOCUMENTO FENIXCMS_7